WEEK 5: Payment Transaction Management System

N.SOWNDARIYA

(E25AI144)

Scenario

The train reservation system allows passengers to make payments for their ticket reservations using different payment methods, such as **UPI, Credit Card, Debit Card, and Cash**. The system stores payment details, including **Payment ID, Reservation ID, Payment Date, Payment Mode, Payment Status, and Transaction Amount**.The system is used to **manage payment transactions**, track **successful, failed, and pending payments**, update the payment status after retries, and generate useful **payment analysis reports** for train reservations.

1. Create Payment Table

```
SQL> CREATE TABLE Payment (
  2       Payment_ID NUMBER,
  3       Order_ID NUMBER,
  4       Payment_Date DATE,
  5       Payment_Mode VARCHAR2(20),
  6       Payment_Status VARCHAR2(20),
  7       Transaction_Amount NUMBER(10,2)
  8  );

Table created.
```

```
SQL> DESC Payment;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PAYMENT_ID                                NOT NULL NUMBER
 ORDER_ID                                  NOT NULL NUMBER
 PAYMENT_DATE                              NOT NULL DATE
 PAYMENT_MODE                              NOT NULL VARCHAR2(20)
 PAYMENT_STATUS                            NOT NULL VARCHAR2(20)
 TRANSACTION_AMOUNT                        NOT NULL NUMBER(10,2)

SQL>
```

2. Inserting Values

```
SQL> INSERT INTO Payment
  2  VALUES (1, 101, TO_DATE('15-SEP-26','DD-MON-RR'), 'UPI', 'Pending', 499);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2  VALUES (2, 102, TO_DATE('16-SEP-26','DD-MON-RR'), 'Card', 'Success', 699);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2  VALUES (3, 103, TO_DATE('17-SEP-26','DD-MON-RR'), 'Cash', 'Success', 299);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2  VALUES (4, 104, TO_DATE('18-SEP-26','DD-MON-RR'), 'Net Banking', 'Pending', 1198);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2  VALUES (5, 105, TO_DATE('19-SEP-26','DD-MON-RR'), 'UPI', 'Success', 799);

1 row created.

SQL>
SQL> INSERT INTO Payment
  2  VALUES (6, 107, TO_DATE('21-SEP-26','DD-MON-RR'), 'Card', 'Pending', 599);

1 row created.
```

3. Viewing Values in Table

SELECT * FROM Payment;

```
SQL> SELECT * FROM Payment;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         1        101 15-SEP-26 UPI                  Success
               499

         2        102 16-SEP-26 Card                 Success
               699

         3        103 17-SEP-26 Cash                 Success
               299


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         4        104 18-SEP-26 Net Banking          Pending
              1198

         5        105 19-SEP-26 UPI                  Success
               799

         6        107 21-SEP-26 Card                 Pending
               599


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         7        108 22-SEP-26 UPI                  Success
              1299
```

```
PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         7        108 22-SEP-26 UPI                  Success
              1299

         8        109 23-SEP-26 Cash                 Success
               749

         9        110 24-SEP-26 Net Banking          Pending
               999


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
        10        111 25-SEP-26 Card                 Success
               449


10 rows selected.
```

4. Establish Relationships

Orders and Payment

• One order can have a payment transaction.

• Each payment transaction is associated with one order.

• The Order_ID in the Payment table is used as a foreign key referencing the Orders table.

Relationship: Orders (1) → Payment (Many)

This relationship allows the system to associate payment transactions with their respective orders.

5. Apply Constraints

The following constraints are applied while creating the table:

• Primary Key:

Payment_ID uniquely identifies each payment transaction.

• Foreign Key: Order_ID references Orders(Order_ID).

• NOT NULL: Essential fields such as Order ID, Payment Date, Payment Mode, Payment Status and Transaction Amount cannot be left empty.

• CHECK Constraint: Payment status is restricted to Completed, Failed or Pending

. • CHECK Constraint: Transaction amount must be greater than zero.

6. Update and Delete Records

Update payment status after retry

```
SQL> UPDATE Payment
  2  SET Payment_Status = 'Success'
  3  WHERE Payment_ID = 11
  4  AND Payment_Status = 'Failed';

1 row updated.

SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_ID = 11;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
        11        111 26-SEP-26 UPI                  Success
               599
```

```
SQL> DELETE FROM Payment
  2  WHERE Payment_ID = 10;

1 row deleted.
```

7. Payment Transaction

Queries REPORT

1. Display All Successful Payments

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Success';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         1        101 15-SEP-26 UPI                  Success
               499

         2        102 16-SEP-26 Card                 Success
               699

         3        103 17-SEP-26 Cash                 Success
               299


PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         5        105 19-SEP-26 UPI                  Success
               799

         7        108 22-SEP-26 UPI                  Success
              1299

         8        109 23-SEP-26 Cash                 Success
               749


6 rows selected.
```

This query displays all successfully completed payment transactions.

REPORT 2. Find Failed Transactions

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Failed';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
        11        111 26-SEP-26 UPI                  Failed
               599
```

This query identifies all failed payment transactions that may require a retry.

REPORT 3. Count Total Successful and Failed Payments

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Failed';

no rows selected

SQL> SELECT Payment_Status, COUNT(*) AS Total_Payments
  2  FROM Payment
  3  WHERE Payment_Status IN ('Success', 'Failed')
  4  GROUP BY Payment_Status;

PAYMENT_STATUS       TOTAL_PAYMENTS
-------------------- --------------
Success                           6

SQL>
```

This query calculates the total number of successful and failed payment transactions.

REPORT 4. Update Failed Payments After Retry

```
SQL> UPDATE Payment
  2  SET Payment_Status = 'Success'
  3  WHERE Payment_ID = 11
  4  AND Payment_Status = 'Failed';

1 row updated.

SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_ID = 11;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
        11        111 26-SEP-26 UPI                  Success
               599
```

This query updates the payment status to Completed when a failed transaction is successfully processed after retry.

REPORT 5. Identify Pending Transactions

```
SQL> SELECT *
  2  FROM Payment
  3  WHERE Payment_Status = 'Pending';

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ---------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
         4        104 18-SEP-26 Net Banking          Pending
              1198

         6        107 21-SEP-26 Card                 Pending
               599

         9        110 24-SEP-26 Net Banking          Pending
               999
```

This query identifies all pending payment transactions that are awaiting confirmation or further processing.

8.Analyze Payment Methods Used by Customers

• Number of transactions using UPI

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM Payment WHERE Payment_Mode = 'UPI';

UPI_TRANSACTIONS
----------------
               5
```

• Number of Card Payments

```
QL> SELECT COUNT(*) AS CARD_PAYMENTS FROM Payment WHERE Payment_Mode IN ('Credit Card','Debit Card');

ARD_PAYMENTS
------------
```

• Most preferred payment method

```
SQL> SELECT Payment_Mode, COUNT(*) AS Transaction_Count FROM Payment GROUP BY Payment_Mode ORDER BY Transaction_Count D
SC;

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- -----------------
UPI                                  5
Credit Card                          4
Debit Card                           2
```

REPORT 2: Revenue Analysis

• Total Revenue generated

```
SQL> SELECT SUM(Transaction_Amount) AS Total_Revenue FROM Payment;

TOTAL_REVENUE
-------------
         9995
```

• Revenue by Payment method

```
SQL> SELECT Payment_Mode, SUM(Transaction_Amount) AS Revenue FROM Payment GROUP BY Payment_Mode ORDER BY Revenue DESC

PAYMENT_MODE           REVENUE
-------------------- ----------
UPI                        4647
Credit Card                2449
Debit Card                 2149
Cash                        750
```

• Average transaction amount

```
SQL> SELECT AVG(Transaction_Amount) AS Average_Transaction_Amount FROM Payment;

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                832.916667
```

REPORT 3: Customer Payment Details

```
SQL> SELECT
  2      C.Customer_Name,
  3      O.Order_ID,
  4      O.Order_Date,
  5      P.Payment_Date,
  6      P.Payment_Mode AS Payment_Mode,
  7      P.Transaction_Amount AS Amount,
  8      P.Payment_Status AS Payment_Status
  9  FROM Customer C
 10  JOIN Orders O
 11      ON C.Customer_ID = O.Customer_ID
 12  JOIN Payment P
 13      ON O.Order_ID = P.Order_ID
 14  ORDER BY P.Payment_Date;

CUSTOMER_NAME                   ORDER_ID ORDER_DAT PAYMENT_D
------------------------------ ---------- --------- ---------
PAYMENT_MODE            AMOUNT PAYMENT_STATUS
-------------------- ---------- --------------------
Arun Kumar                            1001 20-SEP-26 20-SEP-26
UPI                         1299 Completed

Aakash                                1002 21-SEP-26 21-SEP-26
Credit Card                  899 Completed

Rahul Raj                             1003 22-SEP-26 22-SEP-26
Debit Card                  1599 Completed
```

Conclusion

The Payment Transaction Management System helps manage and monitor customer payments. It supports payment record creation, status tracking and updating failed transactions after retries. The five SQL queries help identify successful, failed and pending transactions and provide payment counts for transaction analysis.